

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ОДЕСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ І.І.МЕЧНИКОВА
Факультет математики, фізики та інформаційних технологій Кафедра
математичного забезпечення комп'ютерних систем

КУРСОВА РОБОТА
з дисципліни «Програмування»
СТВОРЕННЯ ІЄРАРХІЇ КЛАСІВ НА ТЕМУ «ВНЗ»
(варіант 13а)

Студента 1 курсу, денна форма навчання
спеціальність F7 «Ком'п'ютерна інженерія»

Зари Кирила Павловича

Керівник: к.т.н., доц. Антоненко О.С.

Національна шкала: _____

Кількість балів: _____

ECTS: _____

Дата захисту: _____

Члени комісії _____ Антоненко О.С.
(підпис) (прізвище та ініціали)

_____ Шестопапов С.В.
(підпис) (прізвище та ініціали)

_____ _____
(підпис) (прізвище та ініціали)

ЗМІСТ

	стор
ВСТУП.....	3
ЗАВДАННЯ	4
1. ТЕОРЕТИЧНА ЧАСТИНА	5
1.1 Поняття ООП.....	5
1.2 Опис об'єктної частини.....	6
2. ПРАКТИЧНА ЧАСТИНА	7
2.1 Опис класів	7
2.2 Інтерфейс користувача	9
ВИСНОВКИ.....	13
СПИСОК ЛІТЕРАТУРИ.....	14

ВСТУП

Мета роботи – створити ієрархію класів на тему «ВНЗ» та застосувати її для розв'язання прикладної задачі, визначеної у варіанті завдання. Під час виконання роботи необхідно використовувати основні принципи об'єктно-орієнтованого програмування: інкапсуляцію, успадкування та поліморфізм.

Згідно із завданням потрібно реалізувати систему взаємопов'язаних класів, що описують структуру та роботу вищого навчального закладу. Необхідно створити конструктори класів, поля та методи для роботи з об'єктами. Поля класів повинні бути закритими, а доступ до них має здійснюватися через відповідні методи класу.

У програмі потрібно реалізувати ієрархію класів на основі відношень успадкування та композиції. Для реалізації динамічного поліморфізму слід використовувати віртуальні методи, абстрактні класи.

ЗАВДАННЯ

Варіант 13а. Створення ієрархії класів на тему «ВНЗ»

У роботі необхідно реалізувати класи, що описують структуру вищого навчального закладу, учасників навчального процесу та їх успішність.

Створити наступну ієрархію класів:

- a. Базовий клас «Людина» (зберігає інформацію про ПІБ, вік та зріст).
- b. Клас-нащадок «Студент» (додатково містить групу, факультет та список оцінок за поточний курс).
- c. Клас-нащадок «Викладач» (додатково містить науковий ступінь та предмет, який він викладає).
- d. Окремий клас «Оцінка» (містить інформацію про предмет, тип контролю: залік, диференційований залік або екзамен, дату).

Для класу «Оцінка» необхідно реалізувати методи переведення балів за шкалою ECTS та оцінку словами («відмінно», «добре», «задовільно» тощо). Також необхідно реалізувати взаємодію між класами та метод для визначення стипендії студента за семестр: алгоритм має повертати підвищену стипендію, якщо всі оцінки «відмінно», та звичайну, якщо немає незадовільних оцінок або незаліків, а середня оцінка не нижче «добре».

1. ТЕОРЕТИЧНА ЧАСТИНА

1.1 Поняття ООП

Об'єктно-орієнтоване програмування – це метод програмування, заснований на поданні програми у вигляді сукупності взаємодіючих об'єктів, кожен з яких є екземпляром певного класу, а класи є членами певної ієрархії наслідування. Програмісти спочатку пишуть клас, а на його основі при виконанні програми створюються конкретні об'єкти (екземпляри класів). На основі класів можна створювати нові, які розширюють базовий клас і таким чином створюється ієрархія класів.

Кожний стиль програмування має свою концептуальну базу. Для ОО стилю такою базою є об'єктна модель. Її побудова основана на використанні 3-х основних концепцій:

- а. Інкапсуляція;
- б. Поліморфізм;
- с. Успадкування.

Інкапсуляція – механізм, який поєднує дані та методи, що обробляють ці дані і захищає і те і інше від зовнішнього впливу або невірного використання. Коли і методи і данні об'єднуються таким чином – створюється об'єкт.

Поліморфізм – властивість, яка дозволяє одне і те саме ім'я використовувати для вирішення декількох технічно різних задач, тобто основною метою поліморфізму є використання одного імені для задання загальних класу дій. На практиці це значить спроможність об'єктів вибирати внутрішній метод або процедуру, виходячи із типу даних, прийнятих в повідомленні

Успадкування – процес, завдяки якому один об'єкт може придбати властивості іншого, тобто наслідувати властивість іншого об'єкту і додавати риси характерні тільки для нього самого.

1.2 Опис об'єктної частини

Програма пропонує систему розподілу та ієрархії для моделювання учасників навчального процесу у вищому навчальному закладі. Можливість виконання тих чи інших дій всередині програми безпосередньо залежить від логіки взаємодії між базовим класом та його спадкоємцями.

Базовим елементом архітектури є клас «Людина», який містить загальні біометричні та ідентифікаційні дані (ПІБ, вік, зріст). Від нього успадковуються два класи: «Студент» та «Викладач», які розширюють базовий функціонал специфічними характеристиками. Викладач інкапсулює інформацію про свій науковий ступінь та профільний предмет, що дозволяє ідентифікувати його роль у ВНЗ.

Студент взаємодіє з системою через свій академічний профіль (належність до факультету та групи), а також через агрегацію списку об'єктів типу «Оцінка». Головною логікою класу «Студент» є метод розрахунку стипендії. Цей метод ітерує через список оцінок поточного курсу студента та реалізує наступну логіку: він перевіряє наявність максимальних балів для призначення підвищеної стипендії, або ж відсутність заборгованостей (незаліків, двійок) за умови середнього балу «добре» для призначення звичайної стипендії.

Клас «Оцінка» виступає окремою сутністю, яка зберігає предмет, тип контрольного заходу та дату його проведення. Важливою частиною цього класу є інкапсульовані методи конвертації балів: один метод відповідає за перетворення внутрішньої числової оцінки в загальноєвропейську шкалу ECTS (A, B, C, D, E, F (незараховано), F (незадовільно)), а інший – за переведення у традиційну текстову шкалу вітчизняних ВНЗ («відмінно», «добре», «задовільно», «незадовільно»). Такий підхід забезпечує зручну взаємодію між класами та виконання основних операцій, пов'язаних із функціонуванням ВНЗ.

2. ПРАКТИЧНА ЧАСТИНА

2.1 Опис класів

Класи у програмі мають наступний функціонал та структуру:

1) Клас «MainSystem»:

а) Приватні поля: Teachers, Students.

б) Конструктор без параметрів

в) Методи:

– clearMemory викликається з деструктора, очищується пам'ять при завершенні роботи програми.

– createNewStudent запускає процедуру створення студента запитуючи дані у користувача.

– createNewTeacher запускає процедуру створення викладача запитуючи дані у користувача.

– showAllTeachers показує список зареєстрованих викладачів у базі даних.

– showAllStudents показує список зареєстрованих студентів у базі даних.

– assignGradeToStudent запускає процедуру виставлення оцінки.

– checkScholaships дозволяє користувачу подивитися список стипендій.

– removePerson дозволяє видалити людину (вчителя або студента за вибором користувача) з бази даних.

– sortStudentsByPerformance показує користувачу список студентів за загальним балом (від великого до малого).

г) Деструктор: ~MainSystem очищує пам'ять при виклику.

2) Абстрактний клас «UniversityPerson»:

а) Приватні поля: fullName, personAge, personHeight.

б) Конструктори:

- Конструктор з параметрами. Приймає дані від користувача.

в) Методи:

- getAge повертає вік людини.
- getHeight повертає зріст людини.
- getName повертає ім'я людини.
- showData віртуальний метод який пише дані на екрані.

г) Деструктор ~ UniversityPerson викликається коли видаляється людина з бази даних.

3) Клас «UniversityStudent», нащадок класу «UniversityPerson»

а) Приватні поля: academicGroup, facultyName, gradeList.

б) Конструктор:

- Конструктор з параметрами

в) Методи:

- addNewGrade дозволяє додати нову оцінку.
- calcAverage повертає середній бал.
- getFaculty повертає назву факультета.
- getGrades повертає вектор оцінки.
- getGroup повертає назву групи.
- getScholashipStatus повертає строку с стану стіпендії.
- showData визначений метод який пише дані у консолі про студента.

4) Клас «UniversityTeacher», нащадок класу «UniversityPerson»

а) Приватні поля: mainSubject, scienceDegree.

б) Конструктор:

- Конструктор з параметрами

в) Методи:

- getDegree повертає наукову ступінь.
- getSubject повертає назву предмета викладача.
- showData визначений метод який пише дані на екрані.

5) Клас «studentGrade»:

а) Приватні поля: gType, points, subjectName

б) Конструктор:

– Конструктор з параметрами

в) Методи:

– convertToEcts повертає строку оцінки по шкалі ECTS.

– getPoints повертає оцінку у форматі числа.

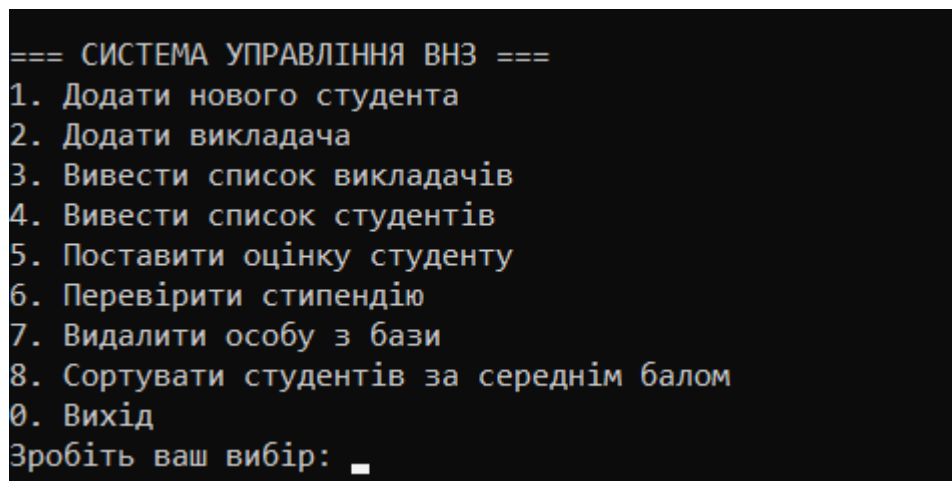
– getSubject повертає строку з назвою дисципліни.

– getType повертає тип оцінки (залік, діф залік, іспит)

– getWordValue повертає оцінку у словісному форматі (добре, відмінно, задовільно, незадовільно)

2.2 Інтерфейс користувача

При запуску програми користувач бачить наступне вікно (Рис. 2.1)



```
=== СИСТЕМА УПРАВЛІННЯ ВНЗ ===
1. Додати нового студента
2. Додати викладача
3. Вивести список викладачів
4. Вивести список студентів
5. Поставити оцінку студенту
6. Перевірити стипендію
7. Видалити особу з бази
8. Сортувати студентів за середнім балом
0. Вихід
Зробіть ваш вибір: _
```

Рисунок 2.1 – початкове вікно

Коли користувач вибирає додати нового студента йому потрібно ввести наступні дані (Рис. 2.2).

```
Зробіть ваш вибір: 1
ПІБ студента: Заря Кирило Павлович
Вік: 21
Зріст (см): 180
Група: f7
Факультет: Мфіт
Студента успішно створено.
```

Рисунок 2.2 – Вікно створення студента

Після вибору додавання викладача користувачу потрібно ввести наступні дані (Рис. 2.3)

```
ПІБ викладача: Андрій Мирошніченко Андрійович
Вік: 20
Зріст (см): 174
Вчений ступінь: Доктор наук
Предмет: Мат.Аналіз
Викладача успішно створено.
```

Рисунок 2.3 – Вікно створення викладача

Після вибору списку в залежності від вибору показує наступні дані (Рис 2.4 та 2.5).

```
--- Зареєстровані особи у базі ---
[1] Андрій Мирошніченко Андрійович, Вік: 20, Зріст: 174 см
    Ступінь: Доктор наук, Дисципліна: Мат.Аналіз
```

Рисунок 2.4 – база даних викладачів

```
--- Зареєстровані особи у базі ---
[1] Заря Кирило Павлович, Вік: 21, Зріст: 180 см
    Факультет: Мфіт, Група: f7
```

Рисунок 2.5 – база даних студентів

Якщо користувач забажає поставити оцінку студенту то йому буде обрати студента та викладача, після цього обрати тип контролю та поставити оцінку від 0-100. Дисципліна оцінки відповідає дисципліни що викладає вчитель.

```
--- Зареєстровані особи у базі ---  
[1] Заря Кирило Павлович, Вік: 21, Зріст: 180 см  
    Факультет: Мфіт, Група: f7  
  
Введіть порядковий номер студента: 1  
  
--- Зареєстровані особи у базі ---  
[1] Андрій Мирошніченко Андрійович, Вік: 20, Зріст: 174 см  
    Ступінь: Доктор наук, Дисципліна: Мат.Аналіз  
  
Введіть порядковий номер викладача: 1  
Оберіть тип контролю (1 - Залік, 2 - Диф. залік, 3 - Іспит):  
1  
Введіть оцінку (0-100): 100  
Оцінка успішно внесена в базу.
```

Рисунок 2.6 – додавання оцінки студенту

Користувач може переглянути стан стипендії у студентів (Рис 2.7 та 2.8)

```
--- Зареєстровані особи у базі ---  
[1] Заря Кирило Павлович, Вік: 21, Зріст: 180 см  
    Факультет: Мфіт, Група: f7  
  
[2] Оліг Дудкін Іванович, Вік: 19, Зріст: 170 см  
    Факультет: МФІТ, Група: АС-163
```

Рисунок 2.7 – список студентів

```
Введіть порядковий номер студента: 1  
Заря Кирило Павлович, Вік: 21, Зріст: 180 см  
    Факультет: Мфіт, Група: f7  
Статус стипендії: Підвищена стипендія  
Успішність:  
- Мат.Аналіз (Залік): 100 балів [А - Зараховано]  
Середній бал: 100
```

Рисунок 2.8 – Статус стипендії

Якщо користувач захоче видалити особу з бази то він спочатку повинен обрати видалити студента чи викладача (Рис 2.9).

```
--- Видалення особи ---  
1. Видалити студента  
2. Видалити викладача  
0. Скасувати  
Ваш вибір: _
```

Рисунок 2.9 – Вибір опції видалення

```
0. Вихід  
Зробіть ваш вибір: 0  
Програма завершує роботу.  
  
C:\Users\Bobot\source\repos\13a\x64\Debug\VNZ.exe (процесс 12300) завершил работу с кодом 0 (0x0).  
Нажмите любую клавишу, чтобы закрыть это окно...
```

Рисунок 2.9 – Вибір опції видалення

```
--- Рейтинг студентів за середнім баллом ---  
1. Заря Кирило Павлович (Група: f7) - Середній бал: 100  
2. Оліг Дудкін Іванович (Група: AC-163) - Середній бал: 0
```

Рисунок 2.10 – рейтинг студентів

Користувач може подивитись список студентів за середнім балом (Рис 2.10)

ВИСНОВКИ

У ході виконання цієї курсової роботи була розроблена програма на C++ яка імітує процеси роботи ВНЗ. Програма була створена за допомогою концепцій об'єктно-орієнтованого програмування, була створена ієрархія класів для людей, що там вчаться та працюють. Також був розроблений клас який дозволяє користувачеві вводити та переглядати дані.

У процесі написання коду програми були використані такі концепції об'єктно-орієнтованого програмування як інкапсуляція, поліморфізм, та спадкування. Були визначені сутності необхідні для імітації роботи ВНЗ, такі як людина, студент, викладач та оцінка. Для кожного з них було створено клас який включає в себе поля, методи, як приватні так і публічні.

У результаті виконання я ознайомився з мовою c++, середою розробки, консольним вводом та виводом та основними концепціями ООП.

СПИСОК ЛІТЕРАТУРИ

1. Прата С. Мова програмування C++. Лекції та вправи. [Текст] / Пер. з англ. — К. : Діалектика, 2021. — 1248 с.
2. Microsoft Learn. Документація з мови C++, конвенцій проектування та керування пам'яттю. [Електронний ресурс]. — Режим доступу: <https://learn.microsoft.com/cpp/cpp/cpp-language-reference>
3. Страуструп Б. Програмування: принципи та практика використання C++. [Текст] / Пер. з англ. — М. : Вільямс, 2020. — 1328 с. cppreference.com. C doxygen-довідник зі стандартної бібліотеки шаблонів STL мови C++. [Електронний ресурс]. — Режим доступу: <https://uk.cppreference.com/>
4. В.О. Коваль, Т.М. Борсук. Об'єктно-орієнтоване програмування та проектування інформаційних систем: навчальний посібник. [Електронне видання] / Одеса : ОНУ, 2025. — 210 с.
5. Репозиторій GitHub на курсову роботу — Режим доступу: <https://github.com/krllzara-uvuvya/VNZ>

Додаток А. Діаграма класів

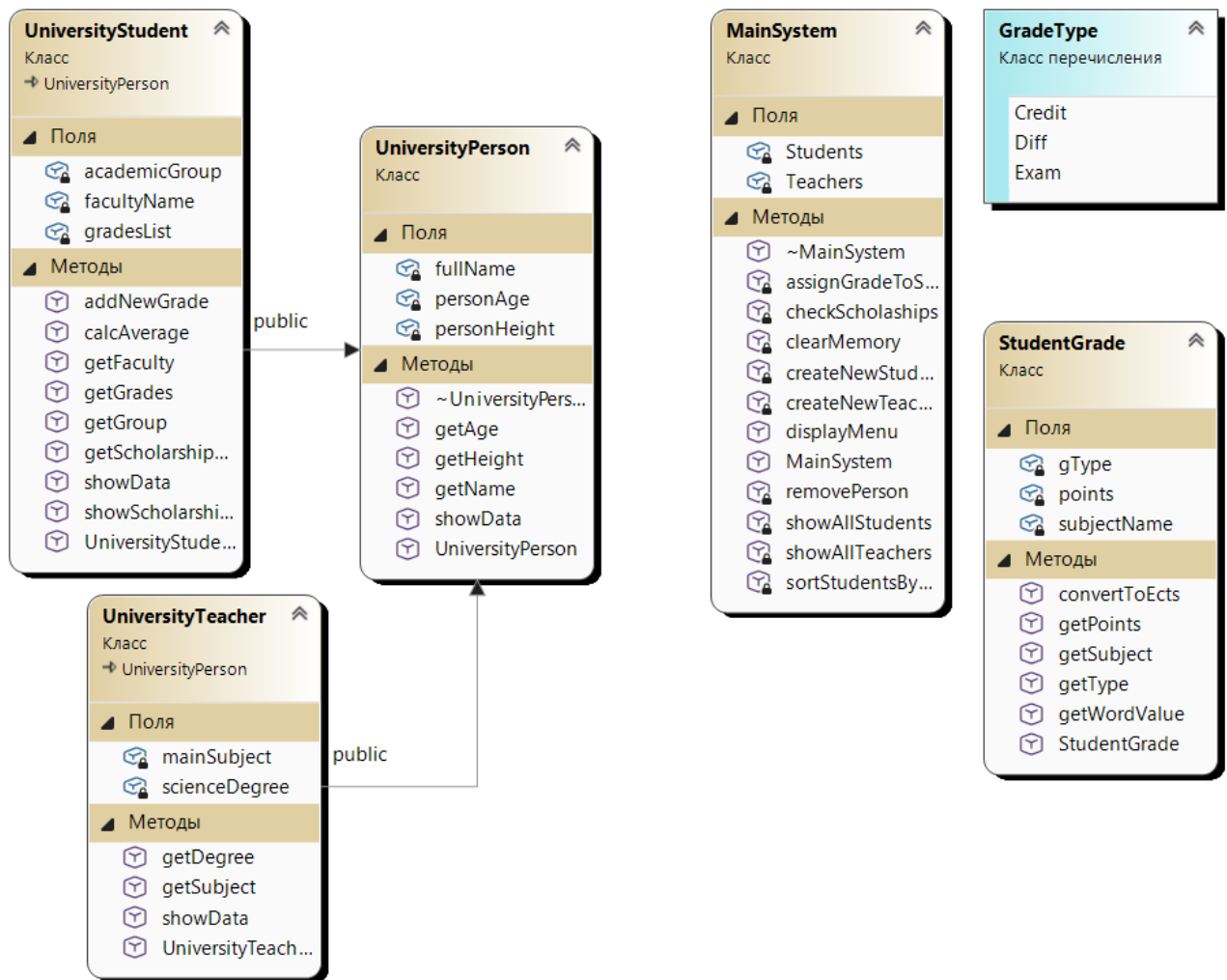


Рисунок А.1 – Діаграма класів

Додаток Б. Код класу

```
#include "MainSystem.h"

void MainSystem::displayMenu() {
    int userChoice = -1;
    while (userChoice != 0) {
        cout << "\n=== СИСТЕМА УПРАВЛІННЯ ВНЗ ===" << endl;
        cout << "1. Додати нового студента" << endl;
        cout << "2. Додати викладача" << endl;
        cout << "3. Вивести список викладачів" << endl;
        cout << "4. Вивести список студентів" << endl;
        cout << "5. Поставити оцінку студенту" << endl;
        cout << "6. Перевірити стипендію" << endl;
        cout << "7. Видалити особу з бази" << endl;
        cout << "8. Сортувати студентів за середнім балом" << endl;
        cout << "0. Вихід" << endl;
        cout << "Зробіть ваш вибір: ";

        if (!(cin >> userChoice)) {
            cout << "Помилка вводу! Спробуйте ще раз." << endl;
            cin.clear();
            cin.ignore(10000, '\n');
            userChoice = -1;
            continue;
        }
        cin.ignore();

        try {
            if (userChoice == 1) createNewStudent();
            else if (userChoice == 2) createNewTeacher();
            else if (userChoice == 3) showAllTeachers();
            else if (userChoice == 4) showAllStudents();
            else if (userChoice == 5) assignGradeToStudent();
            else if (userChoice == 6) checkScholaships();
            else if (userChoice == 7) removePerson();
            else if (userChoice == 8) sortStudentsByPerformance();
            else if (userChoice == 0) cout << "Програма завершує роботу." << endl;
            else cout << "Невірна дія!" << endl;
        }
        catch (const invalid_argument& exception) {
            cout << "\n[ПОМИЛКА ВАЛІДАЦІЇ]: " << exception.what() << " Спробуйте знову." <<
endl;
        }
    }
}

void MainSystem::clearMemory() {
    for (auto teacher : Teachers) delete teacher;
    Teachers.clear();
    for (auto student : Students) delete student;
    Students.clear();
}

void MainSystem::createNewStudent() {
    string name, group, faculty;
    int age, height;
    cout << "ПІБ студента: ";
    getline(cin, name);
    cout << "Вік: ";
    cin >> age;
    cout << "Зріст (см): ";
    cin >> height;
    cin.ignore();
}
```



```

        cout << "Група: ";
        getline(cin, group);
        cout << "Факультет: ";
        getline(cin, faculty);

        Students.push_back(new UniversityStudent(name, age, height, group, faculty));
        cout << "Студента успішно створено." << endl;
    }

    void MainSystem::createNewTeacher() {
        string name, degree, subject;
        int age, height;
        cout << "ПІБ викладача: ";
        getline(cin, name);
        cout << "Вік: ";
        cin >> age;
        cout << "Зріст (см): ";
        cin >> height;
        cin.ignore();
        cout << "Вчений ступінь: ";
        getline(cin, degree);
        cout << "Предмет: ";
        getline(cin, subject);
        Teachers.push_back(new UniversityTeacher(name, age, height, degree, subject));
        cout << "Викладача успішно створено." << endl;
    }

    void MainSystem::showAllTeachers() const {
        if (Teachers.empty()) {
            cout << "База даних вчителів порожня." << endl;
            return;
        }
        cout << "\n--- Зареєстровані особи у базі ---" << endl;
        for (size_t i = 0; i < Teachers.size(); ++i) {
            cout << "[" << i + 1 << "] ";
            Teachers[i]->showData();
            cout << endl;
        }
    }

    void MainSystem::showAllStudents() const {
        if (Students.empty()) {
            cout << "База даних учнів порожня." << endl;
            return;
        }
        cout << "\n--- Зареєстровані особи у базі ---" << endl;
        for (size_t i = 0; i < Students.size(); ++i) {
            cout << "[" << i + 1 << "] ";
            Students[i]->showData();
            cout << endl;
        }
    }

    void MainSystem::assignGradeToStudent() {
        if (Teachers.empty()) {
            cout << "База даних вчителів порожня, нема кому ставити оцінки";
            return;
        }
        showAllStudents();
        if (Students.empty()) return;
        int num;
        cout << "Введіть порядковий номер студента: ";
        cin >> num;
        num--;
    }

```

```

        if (num < 0 || num >= Students.size()) {
            cout << "Помилка: невірний номер!" << endl;
            return;
        }
        UniversityStudent* studentPtr = Students[num];

        showAllTeachers();
        cout << "Введіть порядковий номер викладача: ";
        cin >> num;
        num--;
        if (num < 0 || num >= Teachers.size()) {
            cout << "Помилка: невірний номер!" << endl;
            return;
        }
        UniversityTeacher* teacherPtr = Teachers[num];

        string subject = teacherPtr->getSubject();

        cout << "Оберіть тип контролю (1 - Залік, 2 - Диф. залік, 3 - Іспит): " << endl;
        int typeSelect = 0;

        GradeType finalType = GradeType::Credit;
        while (typeSelect < 1 || typeSelect > 3) {
            cin >> typeSelect;
            if (typeSelect == 1) {
                finalType = GradeType::Credit;
            }
            else if (typeSelect == 2) {
                finalType = GradeType::Diff;
            }
            else if (typeSelect == 3) {
                finalType = GradeType::Exam;
            }
            else {
                cout << "Невідомий тип контролю!" << endl;
            }
        }

        int points;
        cout << "Введіть оцінку (0-100): ";
        cin >> points;
        StudentGrade grade = StudentGrade(subject, finalType, points);
        studentPtr->addNewGrade(grade);
        cout << "Оцінка успішно внесена в базу." << endl;
    }

    void MainSystem::checkScholaships() const {
        showAllStudents();
        if (Students.empty()) return;

        int num;
        cout << "Введіть порядковий номер студента: ";
        cin >> num;
        num--;
        if (num < 0 || num >= (Students.size())) {
            cout << "Помилка: невірний номер!" << endl;
            return;
        }

        UniversityStudent* studentPtr = Students[num];

        studentPtr->showScholarshipData();
    }
}

```

Лістинг Б.1 – Клас MainSystem, аркуш 2

```

void MainSystem::removePerson() {
    cout << "\n--- Видалення особи ---" << endl;
    cout << "1. Видалити студента" << endl;
    cout << "2. Видалити викладача" << endl;
    cout << "0. Скасувати" << endl;
    cout << "Ваш вибір: ";
    int typeChoice;
    cin >> typeChoice;

    if (typeChoice == 0) {
        return;
    }
    else if (typeChoice == 1) {
        showAllStudents();
        if (Students.empty()) return;

        int num;
        cout << "Введіть номер студента для ВИДАЛЕННЯ: ";
        cin >> num;
        num--;
        if (num < 0 || num >= Students.size()) {
            cout << "Помилка: невірний номер!" << endl;
            return;
        }
        delete Students[num];
        Students.erase(Students.begin() + num);
        cout << "Студента успішно видалено з бази даних." << endl;
    }
    else if (typeChoice == 2) {
        showAllTeachers();
        if (Teachers.empty()) return;
        int num;
        cout << "Введіть номер викладача для ВИДАЛЕННЯ: ";
        cin >> num;
        num--;
        if (num < 0 || num >= Teachers.size()) {
            cout << "Помилка: невірний номер!" << endl;
            return;
        }
        delete Teachers[num];
        Teachers.erase(Teachers.begin() + num);
        cout << "Викладача успішно видалено з бази даних." << endl;
    }
    else {
        cout << "Помилка: невірний вибір!" << endl;
    }
}

void MainSystem::sortStudentsByPerformance() {
    vector<UniversityStudent*> students;
    for (auto student : Students) {
        UniversityStudent* s = (student);
        if (s) students.push_back(s);
    }

    if (students.empty()) {
        cout << "У базі немає студентів для сортування." << endl;
        return;
    }

    sort(students.begin(), students.end(), [](const UniversityStudent* first, const
    UniversityStudent* second) {
        return first->calcAverage() > second->calcAverage();
    });

    cout << "\n--- Рейтинг студентів за середнім баллом ---" << endl;
}

```

```

        for (size_t i = 0; i < students.size(); ++i) {
            cout << i + 1 << ". " << students[i]->getName() << " (Група: " << students[i]-
>getGroup()
                << ") - Середній бал: " << students[i]->calcAverage() << endl;
        }
    }
}

```

Лістинг Б.1 – Клас MainSystem, аркуш 3

```

#include "StudentGrade.h"
#include <stdexcept>
StudentGrade::StudentGrade(const string& subject, GradeType gradeType, int gradePoints) {
    if (subject.empty()) throw invalid_argument("Назва предмета не може бути порожньою!");
    if (gradePoints < 0 || gradePoints > 100) throw invalid_argument("Оцінка повинна бути в межах від 0 до 100!");

    subjectName = subject;
    gType = gradeType;
    points = gradePoints;
}

string StudentGrade::getSubject() const { return subjectName; }
GradeType StudentGrade::getType() const { return gType; }
int StudentGrade::getPoints() const { return points; }

string StudentGrade::convertToEcts() const {
    if (points >= 90) return "A";
    if (points >= 85) return "B";
    if (points >= 75) return "C";
    if (points >= 65) return "D";
    if (points >= 60) return "E";
    return (gType == GradeType::Credit) ? "F (незарах.)" : "F (незадов.)";
}

string StudentGrade::getWordValue() const {
    if (gType == GradeType::Credit) {
        return (points >= 60) ? "Зараховано" : "Незараховано";
    }
    if (points >= 90) return "Відмінно";
    if (points >= 75) return "Добре";
    if (points >= 60) return "Задовільно";
    return "Незадовільно";
}

```

Лістинг Б.2 – Клас StudentGrade

```

#include "UniversityPerson.h"
#include <stdexcept>

UniversityPerson::UniversityPerson(const string& name, int age, int height) {
    if (name.empty()) throw invalid_argument("Ім'я не може бути порожнім!");
    if (age <= 0 || age > 150) throw invalid_argument("Некоректний вік!");
    if (height <= 0 || height > 300) throw invalid_argument("Некоректний зріст!");

    fullName = name;
    personAge = age;
    personHeight = height;
}

string UniversityPerson::getName() const {
    return fullName;
}
int UniversityPerson::getAge() const {
    return personAge;
}
int UniversityPerson::getHeight() const {
    return personHeight;
}

```

Лістинг Б.3 – Клас UniversityPerson

```

#include "UniversityStudent.h"
#include <locale>

UniversityStudent::UniversityStudent(const string& name, int age, int height, string group,
string faculty)
    : UniversityPerson(name, age, height)
{
    academicGroup = group;
    facultyName = faculty;
}

string UniversityStudent::getGroup() const { return academicGroup; }
string UniversityStudent::getFaculty() const { return facultyName; }
const vector<StudentGrade>& UniversityStudent::getGrades() const { return gradesList; }

void UniversityStudent::addNewGrade(const StudentGrade& grade) {
    gradesList.push_back(grade);
}

double UniversityStudent::calcAverage() const {
    if (gradesList.empty())
        return 0.0;
    double total = 0;
    for (const auto& grade : gradesList) {
        total += grade.getPoints();
    }
    return total / gradesList.size();
}

string UniversityStudent::getScholarshipStatus() const {
    if (gradesList.empty()) return "Немає оцінок";
    bool onlyExcellent = true;
    bool hasBadGrades = false;

```

Лістинг Б.4 – Клас UniversityStudent

```

        for (const auto& grade : gradesList) {
            if (grade.getPoints() < 60) hasBadGrades = true;
            if (grade.getPoints() < 90) onlyExcellent = false;
        }

        if (onlyExcellent) return "Підвищена стипендія";
        else if (!hasBadGrades && calcAverage() >= 75.0) return "Звичайна стипендія";
        else {
            return "Стипендія не призначена";
        }
    }

}

void UniversityStudent::showData() const {
    cout << getName() << ", Вік: " << getAge() << ", Зріст: " << getHeight() << " см" <<
endl;
    cout << "\t\tФакультет: " << facultyName << ", Група: " << academicGroup << endl;
}

void UniversityStudent::showScholarshipData() const {

    cout << getName() << ", Вік: " << getAge() << ", Зріст: " << getHeight() << " см" <<
endl;
    cout << "\t\tФакультет: " << facultyName << ", Група: " << academicGroup << endl;
    cout << "\t\tСтатус стипендії: " << getScholarshipStatus() << endl;

    if (!gradesList.empty()) {
        cout << "\t\tУспішність:" << endl;
        for (const auto& grade : gradesList) {
            string tStr = (grade.getType() == GradeType::Credit) ? "Залік" :
                (grade.getType() == GradeType::Diff) ? "Диф. залік" : "Іспит";
            cout << "\t\t- " << grade.getSubject() << " (" << tStr << "): "
                << grade.getPoints() << " балів [" << grade.convertToEcts()
                << " - " << grade.getWordValue() << "]" << endl;
        }
        cout << "\t\tСередній бал: " << calcAverage() << endl;
    }
}
}

```

Лістинг Б.4 – Клас UniversityStudent аркуш 2

```

#include "UniversityTeacher.h"

UniversityTeacher::UniversityTeacher(const string& name, int age, int height, string
degree, string subject)
    : UniversityPerson(name, age, height) {
    if (degree.empty()) throw invalid_argument("Ступінь не може бути порожнім!");
    if (subject.empty()) throw invalid_argument("Предмет не може бути порожнім!");
    scienceDegree = degree;
    mainSubject = subject;
}

string UniversityTeacher::getDegree() const { return scienceDegree; }
string UniversityTeacher::getSubject() const { return mainSubject; }

void UniversityTeacher::showData() const {
    cout << getName() << ", Вік: " << getAge() << ", Зріст: " << getHeight() << " см"
<< endl;
    cout << "\t\tСтупінь: " << scienceDegree << ", Дисципліна: " << mainSubject <<
endl;
}

```

Лістинг Б.5 – Клас UniversityTeacher